

Universidad Tecnica Particular de Loja

Ingenieria en Computacion

Programación Orientada a Objetos

Nombre : Geovanny Sebastian Codena Gonzalez

Fecha: 24/07/2026

Informe

Nota importante: El proyecto lo realice de un integrante debido al preguntar a mis compañeros si alguno podía trabajar conmigo y me supieron decir que ya tenia grupo por lo cual me toco realizarlo de manera individual para así cumplir con el trabajo.

Sistema de Gestión de Activos de Laboratorio (Lab-Inventario)

El presente proyecto consiste en desarrollar un sistema “Lab-Inventario”, el cual este desarrollado bajo el lenguaje de programación “Java” para la gestión de activos tecnológicos pertenecientes a un laboratorio universitario.

El sistema permite registrar, consultar, actualizar y eliminar activos como hardware, periféricos y licencias, además de generar reportes automáticos del valor de los activos y costos de mantenimiento preventivo.

Para garantizar un código sostenible y escalable se aplicaron principios SOLID arquitectura MVC, persistencia mediante SQLite y programación orientada a objetos.

Análisis del problema

La administración Manual de equipos tecnológicos puede provocar perdida de información, dificultad para conocer el estado de activos y problemas para calcular cotos de mantenimiento.

LabInventario permite centralizar la información de los activos, facilitando su seguimiento calcular los costos de mantenimiento y la toma de decisiones mediante reportes.

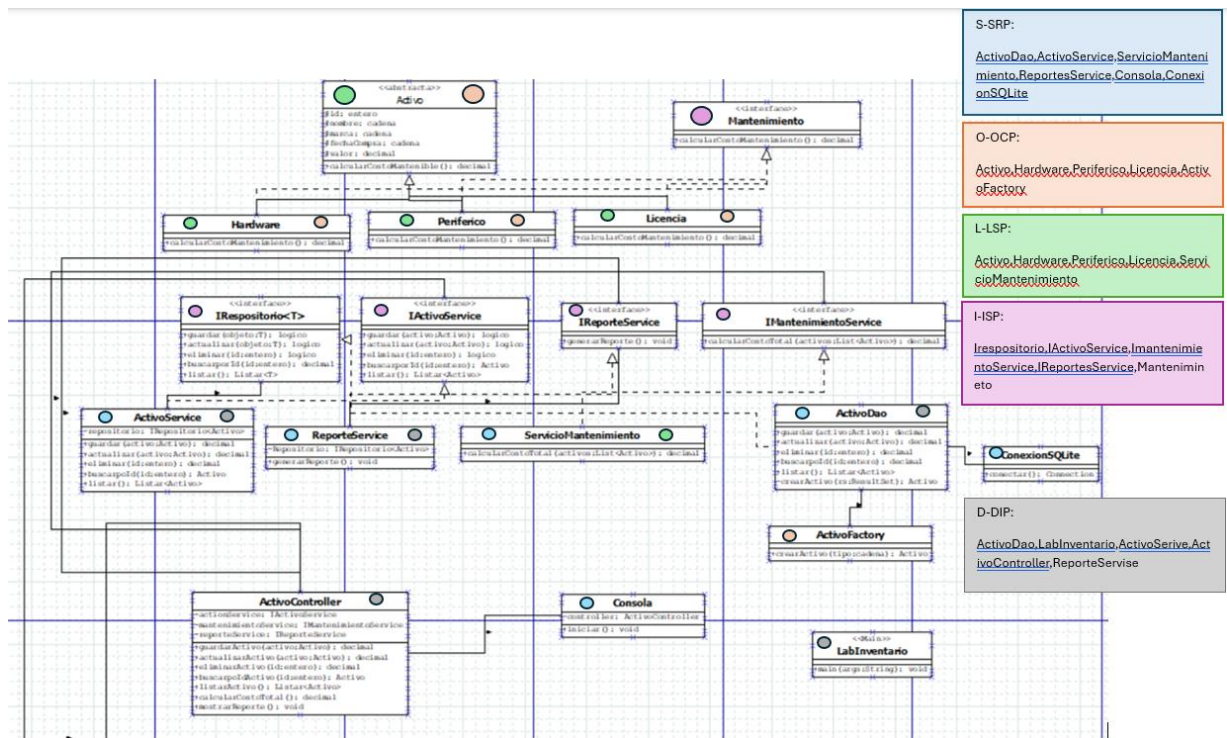
Objetivos

Desarrollar un sistema de gestión de activos de laboratorio utilizando Java, aplicando arquitectura MVC, principios SOLID y una base de datos SQLite.

Objetivos específicos

- Implementar operaciones CRUD para activos.
- Aplicar herencia y polimorfismo.
- Separar responsabilidades mediante capas.
- Implementar persistencia con JDBC y SQLite.
- Generar reportes de mantenimiento.
- Realizar pruebas unitarias.

Diseño del sistema



El sistema fue diseñado utilizando una arquitectura MVC donde los modelos representan los activos, los controladores gestiona la interacción con el usuario y los servicios contiene la lógica del negocio.

Arquitectura MVC

Modelo

Este Paquete contiene la clase Abstracta Activo y sus especializaciones Hardware, Periférico y licencia. Esta clase implementa polimorfismo mediante método calcularCostoMantenimiento().

Vista

Aquí se encuentra el menú “Consola” el cual corresponde al menú de interacción donde el usuario puede seleccionar las operaciones disponibles.

Controlador

ActivoController recibe las solicitudes del usuario y comunica la vista con los servicios mediante interfaces, evitando dependencia directa con las implementaciones.

Aplicación de principios SOLID

SRP (Single Responsibility Principle)

¿Qué dice este principio?

Una clase debe tener una sola razón para cambiar, es decir, una clase no debe encargarse de muchas cosas diferentes.

En el sistema de inventario se separaron las responsabilidades en diferentes capas siguiendo la arquitectura MVC.

La estructura queda así:

MODELO

|-- Activo

|-- Hardware

|-- Periferico

|-- Licencia

DAO

|-- ActivoDao

SERVICE

```
|-- IActivoService  
|-- ActivoService  
|-- ReporteService
```

CONTROLLER

```
|-- ActivoController
```

¿Qué responsabilidad tiene cada parte?

1. Modelos (Modelo)

Los modelos solamente representan la información del activo.

Ejemplo:

Clase:

Activo

Responsabilidad:

- Guardar código del activo.
- Guardar nombre.
- Guardar estado.
- Guardar costo.
- Guardar fecha de adquisición.

Esta clase NO debe:

- Insertar datos en SQLite.
- Generar reportes.
- Mostrar información en pantalla.

Porque esas responsabilidades pertenecen a otras clases.

2. DAO (Data Access Object)

El DAO solamente se encarga de la comunicación con la base de datos.

ActivoDao

Responsabilidad:

- Insertar activos.
- Consultar activos.
- Actualizar activos.
- Eliminar activos.

El DAO no debe:

- Calcular mantenimientos.
- Decidir reglas
- Mostrar información.

3. Services (Servicios)

Aquí están las reglas del programa.

Ejemplo:

ReporteService

Responsabilidad:

- Crear reportes.
- Calcular estadísticas.
- Aplicar reglas.

4. Controller

El controlador coordina las acciones.

ActivoController

Responsabilidad:

- Recibir solicitudes de la interfaz.
- Llamar servicios.
- Devolver resultados.

Resultado del principio SRP

Cada clase tiene una única responsabilidad:

Clase	Responsabilidad
Activo	Representar un activo
Hardware	Representar activos hardware
ActivoDao	Comunicación SQLite
ReporteService	Generar reportes
ActivoController	Controlar operaciones

OCP (Open/Closed Principle)

El cálculo de costos de mantenimiento está abierto a extensión, cerrado a modificación: agregar un nuevo tipo de activo implica únicamente crear una nueva subclase de Activo que implemente `calcularCostoMantenimiento()`. Ningún servicio (`ServicioMantenimiento`, `ReporteService`) necesita modificarse, porque ambos operan sobre la referencia `Activo` y llaman siempre al mismo método polimórfico.

Observación honesta: `ActivoFactory.crearActivo(String tipo)` sí usa un switch por tipo para instanciar la clase concreta correcta al leer desde la base de datos. Esto es una limitación conocida y aceptada de Java sin reflexión/frameworks (alguien tiene que decidir qué new ejecutar): agregar un tipo nuevo implica tocar solo esa fábrica, no la lógica de negocio (`ServicioMantenimiento`, `ReporteService`, `ActivoController`), que es donde realmente importa que OCP se cumpla.

LSP (Liskov Substitution Principle)

`ServicioMantenimiento.calcularCostoTotal(List<Activo> activos)` y `ReporteService.generarReporte()` recorren la lista y llaman a `activo.calcularCostoMantenimiento()` sin ningún instanceof ni cast a `Hardware`, `Periferico` o `Licencia`. Cualquier subclase de `Activo` puede sustituir a la clase base sin romper el comportamiento correcto del programa, validado explícitamente en `ServicioMantenimientoTest`, que mezcla los tres tipos en una misma lista y obtiene el total correcto .

ISP (Interface Segregation Principle)

En vez de una única interfaz gigante, el proyecto define cuatro interfaces pequeñas y específicas:

IRespositorio<T>: contrato genérico mínimo de persistencia (guardar, actualizar, eliminar, buscarPorId, listar).

IActivoService: operaciones CRUD de activos.

IMantenimientoService: únicamente calcularCostoTotal(List<Activo>).

IReporteService: únicamente generarReporte().

Adicionalmente, Mantenimiento (en el paquete interfaces) es una interfaz de un solo método (calcularCostoMantenimiento()) que Activo implementa, un "role interface" que expone exactamente la capacidad que otras clases necesitan, nada más.

Ningún cliente depende de métodos que no usa: ActivoController recibe IActivoService, IMantenimientoService e IReporteService por separado, no una única interfaz con los tres bloques de responsabilidad mezclados.

DIP (Dependency Inversion Principle)

Las dependencias siempre apuntan hacia abstracciones:

ActivoController --> IActivoService, IMantenimientoService, IReporteService (interfaces)

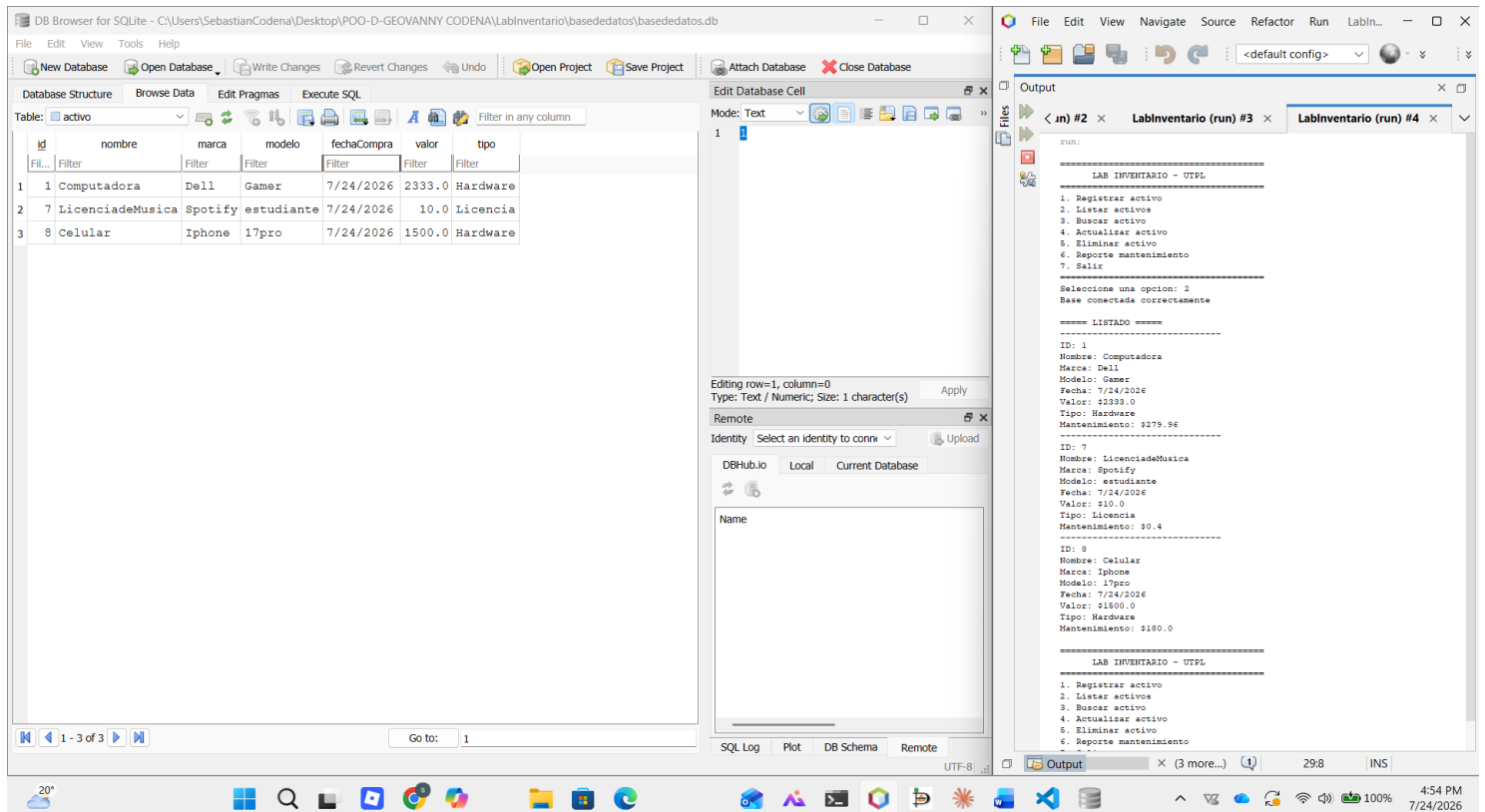
ActivoService --> IRespositorio<Activo> (interfaz)

ReporteService --> IRespositorio<Activo> (interfaz)

Ninguna clase de controlador o servicio importa ActivoDao directamente. La única clase de todo el proyecto que menciona la implementación concreta es LabInventario.java, que arma el grafo de objetos e inyecta las dependencias por constructor.

Persistencia SQLite

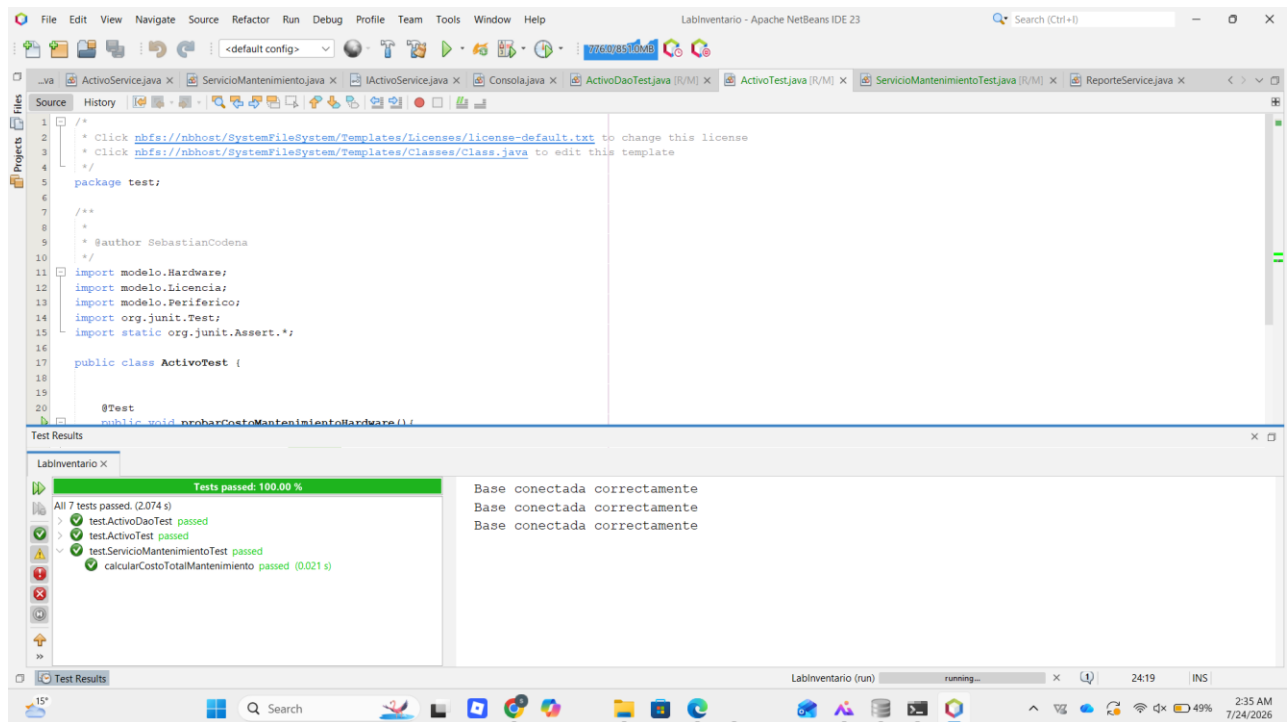
La información se almacena mediante SQLite utilizando JDBC. La clase ConexionSqlite administra la conexión y los DAO realizan las operaciones CRUD



Pruebas unitarias

Se realizaron pruebas unitarias utilizando JUnit para verificar:

- Cálculo de mantenimiento de cada tipo de activo.
- Registro de activos mediante DAO.
- Consulta y listado de activos.
- Cálculo total de mantenimiento.



Uso de GitHub

El desarrollo fue gestionado mediante Git utilizando ramas para organizar funcionalidades y commits descriptivos para registrar los cambios realizados.

```
PS C:\Users\SebastianCodena\Desktop\P00-D-GEOVANNY CODENA\LabInventario> git log
commit b2d3356225f47245dcfb5384897a014b66a93f47 (HEAD -> feature/LbInventario, origin/feature/LbInventario)
Author: Sebastian <codenageovanny99@gmail.com>
Date: Fri Jul 24 11:53:47 2026 -0500

    Correcion solid

commit b45effb048ed08cb79a6d00304338d7be2765007
Author: Sebastian <codenageovanny99@gmail.com>
Date: Fri Jul 24 03:03:59 2026 -0500

    Correcion Solid

commit b78ee44792c937487f052855c2a38df23780c659
Author: Sebastian <codenageovanny99@gmail.com>
Date: Fri Jul 24 02:36:36 2026 -0500

    ejecucion de test

commit 194710010c4a09b314c214feb4d97654bald6ada
Author: Sebastian <codenageovanny99@gmail.com>
Date: Fri Jul 24 01:40:21 2026 -0500

    correcion de errores

commit 8be3dab378ea79c8d20bb897884e2c3e78081cba
Author: Sebastian <codenageovanny99@gmail.com>
Date: Fri Jul 24 00:34:25 2026 -0500

    Implementacion de reportes de activos desde SQLite
```

```

commit 95e87b7305d9a085c4a704cd62dbc42b9f736b97
Author: Sebastian <codenageovanny99@gmail.com>
Date:   Fri Jul 24 00:21:32 2026 -0500

    Implementacion de los test y el main para la ejecucion del programa

commit 9af9bb5d81cf6c00922259030b07155aee3288cb
Author: Sebastian <codenageovanny99@gmail.com>
Date:   Fri Jul 24 00:14:42 2026 -0500

    Implementacion de la vista

commit b8f9c3ef7c11381cfbea6f7c66fb404795a1e8f4
Author: Sebastian <codenageovanny99@gmail.com>
Date:   Fri Jul 24 00:08:26 2026 -0500

    Implementacion de controller y servicios

commit 5e8d764cb70cf10f583844e366793973898e1ebe
Author: Sebastian <codenageovanny99@gmail.com>
Date:   Fri Jul 24 00:00:25 2026 -0500

    Implementacion de la conexionSQLite

commit d7361ce26f386c95ac5c783107e65f7ffa32d7f3 (origin/main, main)
Author: Sebastian <codenageovanny99@gmail.com>
Date:   Thu Jul 23 23:52:49 2026 -0500

    first commit
:|

```

Códigos Importantes

Polimorfismo puro en la lógica de negocio (LSP/OCP)

ReporteService.java

```

for (Activo activo : activos) {

    double mantenimiento =
activo.calcularCostoMantenimiento(); // sin instanceof

    ...

}

```

Explicación:

Se aplica polimorfismo porque todas las clases que heredan de Activo utilizan el mismo método, pero cada una implementa su propio cálculo. No se necesitan validaciones como instanceof, permitiendo agregar nuevos tipos de activos sin modificar la lógica existente.

Factory Method para la creación de activos (OCP acotado a un solo punto)

ActivoFactory.java

```

public static Activo crearActivo(String tipo) {

    switch (tipo) {

        case "Hardware":    return new Hardware();

        case "Periferico":  return new Periferico();

        case "Licencia":    return new Licencia();

    }
}

```

```

        default: throw new IllegalArgumentException("Tipo
de activo no válido.");
    }
}

```

Explicación:

La fábrica centraliza la creación de objetos Activo, evitando que otras clases tengan que conocer las clases concretas. Si se agrega un nuevo tipo de activo, solamente se modifica esta fábrica y no los servicios principales.

Inversión de dependencias por constructor (DIP)

LabInventario.java (composition root)

```

IRespositorio<Activo> repositorio = new ActivoDao();

IActivoService activoService = new
ActivoService(repositorio);

ActivoController controller = new
ActivoController(activoService, mantenimientoService,
reporteService);

```

Explicación:

Las clases dependen de interfaces en lugar de clases concretas. Esto permite cambiar implementaciones, como el sistema de almacenamiento, sin modificar la lógica de negocio.

Interfaz genérica de persistencia (ISP + reutilización)

```

public interface IRespositorio<T> {

    boolean guardar(T objeto);

    boolean actualizar(T objeto);

    boolean eliminar(int id);

    T buscarPorId(int id);

    List<T> listar();

}

```

Explicación:

Esta interfaz define las operaciones básicas que debe tener cualquier repositorio.

Al ser genérica puede reutilizarse para diferentes entidades y evita crear interfaces demasiado grandes con métodos innecesarios.

CRUD completo con JDBC (extracto)

```
@Override

public boolean guardar(Activo activo) {

    String sql = "INSERT INTO
activo(nombre,marca,modelo,fechaCompra,valor,tipo)
VALUES(?,?,?, ?, ?, ?) ";

    try (Connection con = ConexionSQLite.conectar());
        PreparedStatement ps = con.prepareStatement(sql))
    {

        ps.setString(1, activo.getNombre());
        ps.setString(2, activo.getMarca());
        ps.setString(3, activo.getModelo());
        ps.setString(4, activo.getFechaCompra());
        ps.setDouble(5, activo.getValor());
        ps.setString(6, activo.getClass().getSimpleName());

        return ps.executeUpdate() > 0;
    } catch (SQLException e) {

        System.out.println(e.getMessage());

    }

    return false;
}
```

Explicación:

Este método implementa la operación de guardar del CRUD utilizando JDBC y SQLite. Toma los datos del objeto Activo y los inserta en la base de datos mediante PreparedStatement, manteniendo separada la gestión de datos de la lógica del sistema.

Conclusiones

El proyecto LabInventario cumple con los objetivos planteados al desarrollar un sistema de gestión de activos de laboratorio que permite centralizar la

información de los equipos tecnológicos, facilitar su seguimiento y automatizar el cálculo de costos de mantenimiento. La implementación de operaciones CRUD, persistencia mediante JDBC con SQLite y generación de reportes permite mejorar la administración de los activos y apoyar la toma de decisiones.

Además, el sistema aplica correctamente la arquitectura MVC y una separación por capas (modelo → dao → servicio → controlador → vista), logrando una mejor organización del código y distribución de responsabilidades. La aplicación de los principios SOLID permite obtener un software más flexible y mantenible: se utiliza herencia y polimorfismo para el manejo de diferentes tipos de activos, interfaces específicas para evitar dependencias innecesarias y abstracciones para facilitar la extensión del sistema.

Finalmente, mediante las pruebas unitarias se verifica el correcto funcionamiento de la lógica principal, especialmente el cálculo de mantenimiento y el comportamiento de los diferentes tipos de activos. En conjunto, LabInventario proporciona una solución escalable y organizada para la gestión de equipos tecnológicos, reduciendo problemas de pérdida de información y facilitando el control eficiente del mantenimiento de los activos del laboratorio.